Regex in Python

Computation and Discrete Structures III

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Department of Computing and Intelligent Systems
Barberi Faculty of Engineering, Design and Applied Sciences
ICESI University

Material adapted from DataCamp - Regular expressions in Python

2026

Contents

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

Contenido

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

What is a regular expression?

Definition

REGular EXpression or **regex**:
A string containing a combination of normal characters and special metacharacters that describes patterns to find text or positions within a text.

What is a regular expression?

Definition

REGular EXpression or **regex**:
A string containing a combination of normal characters and special metacharacters that describes patterns to find text or positions within a text.

Normal characters match themselves (`st`)

What is a regular expression?

Definition

REGular EXpression or **regex**:
A string containing a combination of normal characters and special metacharacters that describes patterns to find text or positions within a text.

Metacharacters represent types of characters (`\d`, `\s`, `\w`) or ideas (`{3,10}`)

- `\d` matches any digit (0–9)
- `\s` matches any whitespace character
- `\w` matches any word character (letters, digits, underscore)

What is a regular expression?

Definition

REGular EXpression or **regex**:
A string containing a combination of normal characters and special metacharacters that describes patterns to find text or positions within a text.

Pattern: a sequence of characters that maps to words or punctuation

- Patterns define what we're searching for
- Can be simple (literal text) or complex (with metacharacters)

What is a regular expression?

Pattern matching usage

- Find and replace text
- Validate strings (emails, phone numbers, passwords)
- Extract information from text
- Very powerful and fast

Common applications:

- Data cleaning and preprocessing
- Text parsing and extraction
- Input validation
- Search and replace operations

Contenido

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

The re module

```python
import re
```

Find all matches of a pattern:

```python
re.findall(r"#movies",
           "Love #movies! I had fun yesterday going to the #movies")
```

Output:

```
['#movies', '#movies']
```

Note: The r prefix creates a raw string, preventing Python from interpreting backslashes.

The re module

```python
import re
```

Split string at each match:

```python
re.split(r"!",
         "Nice Place to eat! I'll come back! Excellent meat!")
```

Output:

```
['Nice Place to eat', " I'll come back", ' Excellent meat', '']
```

Note: The empty string at the end appears because there's nothing after the last exclamation mark.

The re module

```python
import re
```

Replace one or many matches with a string:

```python
re.sub(r"yellow", "nice",
       "I have a yellow car and a yellow house in a yellow
          neighborhood")
```

Output:

```
'I have a nice car and a nice house in a nice neighborhood'
```

Note: `re.sub()` replaces ALL occurrences by default.

Contenido

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

Supported metacharacters

Digit metacharacters:

- `\d` matches any digit (0–9)
- `\D` matches any non-digit character

```
re.findall(r"User\d", "The winners are: User9, UserN, User8")
```

Output: `['User9', 'User8']`

```
re.findall(r"User\D", "The winners are: User9, UserN, User8")
```

Output: `['UserN']`

Supported metacharacters

Word character metacharacters:

- `\w` matches word characters (letters, digits, underscore)
- `\W` matches non-word characters

```
re.findall(r"User\w", "The winners are: User9, UserN, User8")
```

Output: `['User9', 'UserN', 'User8']`

```
re.findall(r"\W\d", "This skirt is on sale, only $5 today!")
```

Output: `['$5']`

Supported metacharacters

Whitespace metacharacters:

- \s matches whitespace (space, tab, newline)
- \S matches non-whitespace characters

```
re.findall(r"Data\sScience", "I enjoy learning Data Science")
```

Output: `['Data Science']`

```
re.sub(r"ice\Scream", "ice cream", "I really like ice-cream")
```

Output: `'I really like ice cream'`

Let’s practice!

Contenido

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

Repeated characters

Challenge: Validate the following string pattern:

`password1234`

Requirements:

- 8 word characters followed by
- 4 digits

We need a way to express repetition without writing the same metacharacter multiple times!

Repeated characters

Without quantifiers (inefficient):

```python
import re
password = "password1234"
re.search(r"\w\w\w\w\w\w\w\w\d\d\d\d", password)
```

Output:

```
<_sre.SRE_Match object; span=(0, 12), match='password1234'>
```

This works but is verbose and hard to read!

Repeated characters

With quantifiers (efficient):

```python
import re
password = "password1234"
re.search(r"\w{8}\d{4}", password)
```

Output:

```
<_sre.SRE_Match object; span=(0, 12), match='password1234'>
```

Quantifiers

A metacharacter that tells the regex engine how many times to match a character immediately to its left.

Quantifiers

Once or more: +

```
text = "Date of start: 4-3. Date of registration: 10-04."
re.findall(r"        ", text)
```

What pattern should we use to match dates in format `d-d` *or* `dd-dd`*?*

Quantifiers

Once or more: +

```
text = "Date of start: 4-3. Date of registration: 10-04."
re.findall(r"\d+-   ", text)
```

Now we match one or more digits, followed by a hyphen. We need to complete the pattern...

Quantifiers

Once or more: +

```
text = "Date of start: 4-3. Date of registration: 10-04."
re.findall(r"\d+-\d+", text)
```

Output: `['4-3', '10-04']`

The + quantifier matches one or more occurrences of the preceding element.

Quantifiers

Zero times or more: $*$

```
my_string = "The concert was amazing! @ameli!a @joh&&n @mary90"
re.findall(r"@\w+\W*\w+", my_string)
```

Output: `['@ameli!a', '@joh&&n', '@mary90']`

This pattern matches usernames that may contain special characters in the middle. The $*$ allows zero or more non-word characters between word segments.

Quantifiers

Zero times or once: ?

```
text = "The color of this image is amazing. However, the colour blue could be brighter."
re.findall(r"colou?r", text)
```

Output: `['color', 'colour']`

The ? makes the preceding character optional. This is useful for handling American vs British spelling variations.

Quantifiers

n times at least, m times at most: `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
re.findall(r"                                ", phone_number)
```

Let's build a pattern step by step to match phone numbers with varying formats...

Quantifiers

n times at least, m times at most: `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
re.findall(r"\d{1,2}-                                  ", phone_number)
```

Now we match 1–2 digits followed by a hyphen (country code part)...

Quantifiers

n times at least, m times at most: `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
re.findall(r"\d{1,2}-\d{3}-            ", phone_number)
```

Adding the area code (3 digits)...

Quantifiers

n times at least, m times at most: `{n, m}`

```
phone_number = "John: 1-966-847-3131 Michelle: 54-908-42-42424"
re.findall(r"\d{1,2}-\d{3}-\d{2,3}-\d{4,}", phone_number)
```

Output: `['1-966-847-3131', '54-908-42-42424']`

Complete pattern: country code (1–2 digits), area code (3 digits), exchange (2–3 digits), and number (4+ digits).

Quantifiers

Important Note

Immediately to the left

In the pattern `r"apple+"`, the + applies to e and not to `apple`

To apply to the entire word, use grouping: `r"(apple)+"`

Examples:

- `apple+` matches: apple, applee, appleee
- `(apple)+` matches: apple, appleapple, appleappleapple

Let’s practice!

Contenido

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

Looking for patterns

Two different operations to find a match:

`re.search()` - searches anywhere in the string:

```
re.search(r"\d{4}", "4506 people attend the show")
```

Output: `<...Match object; span=(0, 4), match='4506'>`

```
re.search(r"\d+", "Yesterday, I saw 3 shows")
```

Output: `<...Match object; span=(17, 18), match='3'>`

Looking for patterns

`re.match()` - searches only at the beginning:

```
re.match(r"\d{4}", "4506 people attend the show")
```

Output: `<...Match object; span=(0, 4), match='4506'>`

```
re.match(r"\d+", "Yesterday, I saw 3 shows")
```

Output: `None`

Key Difference

`re.search()` finds a match anywhere; `re.match()` only at the start

Special characters

Match any character (except newline): .

```
my_links = "Just check out this link: www.amazingpics.com. It has amazing photos!"
re.findall(r"www  com", my_links)
```

The dot . is a wildcard that matches any single character. What pattern should we use?

Special characters

Match any character (except newline): `.`

```
my_links = "Just check out this link: www.amazingpics.com. It has amazing photos!"
re.findall(r"www.+com", my_links)
```

Output: `['www.amazingpics.com']`

The pattern `.+` *means "one or more of any character". This matches everything between* `www` *and* `com`*.*

Special characters

Start of the string: ^

```
my_string = "the 80s music was much better that the 90s"
re.findall(r"the\s\d+s", my_string)
```

Output: `['the 80s', 'the 90s']`

```
re.findall(r"^the\s\d+s", my_string)
```

Output: `['the 80s']`

The ^anchor restricts the match to the beginning of the string only.

Special characters

End of the string: $

```
my_string = "the 80s music hits were much better that the 90s"
re.findall(r"the\s\d+s$", my_string)
```

Output: `['the 90s']`

The $ anchor restricts the match to the end of the string. Together with ^, you can match entire strings precisely.

Special characters

**Escape special characters: **

```
my_string = "I love the music of Mr.Go. However, the sound was too
    loud."
print(re.split(r".\s", my_string))
```

Output: `['', 'lov', 'th', 'musi', 'o', 'Mr.Go', 'However', 'th', 'soun', 'wa', 'to', 'loud.']`

Without escaping, the dot matches ANY character. This splits at every character followed by space!

Special characters

Escape special characters: \

```
my_string = "I love the music of Mr.Go. However, the sound was too
    loud."
print(re.split(r"\.\s", my_string))
```

Output: `['I love the music of Mr.Go', 'However, the sound was too loud.']`

By escaping the dot with \., we match literal periods only.

Other characters that need escaping: `. * + ? [ ] { } ( ) ^ $ |`

OR operator

Character: |

```
my_string = "Elephants are the world's largest land animal! I would
    love to see an elephant one day"
re.findall(r"Elephant|elephant", my_string)
```

Output: `['Elephant', 'elephant']`

The pipe | acts as a logical OR, matching either pattern on its left or right.

OR operator

Set of characters: `[ ]`

```
my_string = "Yesterday I spent my afternoon with my friends:
    MaryJohn2 Clary3"
re.findall(r"[a-zA-Z]+\d", my_string)
```

Output: `['MaryJohn2', 'Clary3']`

Character sets `[a-zA-Z]` match any letter (lowercase or uppercase). Ranges make it easy to specify multiple characters.

OR operator

Set of characters: `[ ]`

```
my_string = "My&name&is#John Smith. I%live$in#London."
re.sub(r"[#$%&]", " ", my_string)
```

Output: `'My name is John Smith. I live in London.'`

Character sets are perfect for matching multiple specific characters. This replaces any of the special characters with a space.

OR operator

Negated set of characters: `[^...]`

```
my_links = "Bad website: www.99.com. Favorite site: www.hola.com"
re.findall(r"www[^0-9]+com", my_links)
```

Output: `['www.hola.com']`

Important

Inside character sets, ^ at the beginning negates the set. It means "match anything EXCEPT these characters".

This pattern matches URLs that don't contain digits between `www` and `com`.

Let’s practice!

Contenido

1. What is a Regular Expression?

2. The re Module

3. Supported Metacharacters

4. Repetitions

5. Regex Metacharacters

6. Greedy vs. Non-greedy Matching

Greedy vs. non-greedy matching

Two types of matching methods:

- **Greedy** (default behavior)
- **Non-greedy** or lazy

Standard Quantifiers

Standard quantifiers are greedy by default: `*`, `+`, `?`, `{num, num}`

Understanding the difference is crucial for writing precise patterns!

Greedy matching

Greedy: match as many characters as possible
Return the longest match

```python
import re
re.match(r"\d+", "12345bcada")
```

Output: `<...Match object; span=(0, 5), match='12345'>`

The + quantifier is greedy: it matches all consecutive digits, not just one.

Greedy matching

Backtracks when too many characters matched

Gives up characters one at a time

```
import re
re.match(r".*hello", "xhelloxxxxxx")
```

Output: `<...Match object; span=(0, 6), match='xhello'>`

The `.*` initially matches everything, then backtracks until it can match `hello`.

Non-greedy matching

Lazy: match as few characters as needed
Returns the shortest match
Append ? to greedy quantifiers

```python
import re
re.match(r"\d+?", "12345bcada")
```

Output: `<...Match object; span=(0, 1), match='1'>`

The +? is non-greedy: it matches the minimum (just one digit).

Non-greedy matching

Backtracks when too few characters matched
Expands characters one at a time

```python
import re
re.match(r".*?hello", "xhelloxxxxxx")
```

Output: `<...Match object; span=(0, 6), match='xhello'>`

The `.*?` matches as little as possible while still allowing `hello` to match.

Common non-greedy quantifiers: `*?`, `+?`, `??`, `{n,m}?`

Let’s practice!

Summary

Key concepts covered:

- Regular expressions are powerful tools for pattern matching
- Python's `re` module provides functions: `findall()`, `search()`, `match()`, `split()`, `sub()`
- Metacharacters: `|, |, | and their negations`
- `Quantifiers: +, *, ?, {n,m}`
- `Special characters: ., ^, $, |, [ ]`
- `Greedy vs non-greedy matching`

Practice is essential! Experiment with different patterns to master regex.